

Tutorial Sheet 1

An Introduction to Optimal Control

Thulasi Mylvaganam

Department of Aeronautics, Imperial College London

Exercise 1 Consider a simplified model of a rigid spacecraft rotating about a principal axis. The rotational dynamics are given by

$$\dot{x}(t) = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ \frac{1}{M_I} \end{bmatrix} u(t),$$

where M_I is the spacecraft's moment of inertia, $x = [x_1 \ x_2]^\top$ is the state of the system, with $x_1(t) = \theta(t)$ representing the angular deviation from the desired orientation and $x_2(t) = \dot{\theta}(t)$ represents the angular velocity, and the control input $u(t)$ is the torque applied by spacecraft's thrusters.

Suppose, we wish to design the control input u so that the spacecraft is aligned with a desired orientation corresponding to $\theta = 0$, while minimising fuel consumption.

Consider a finite-horizon optimal control problem with the cost

$$J = \int_0^T (x(t)^\top Q x(t) + u(t)^\top R u(t)) dt + x(T)^\top Q_T x(T).$$

(a) Discuss how the matrices Q , R and Q_T should be assigned to:

- strongly penalise orientation error;
- moderately penalise angular velocity (which we would like to approach zero);
- Lightly penalise fuel usage.

(b) Discuss how the matrices should be tuned if the main concern is that the spacecraft has very limited fuel left (and exhausting the available fuel will result in system failure).

(a)

$$Q = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix}, \quad R = \begin{bmatrix} r_1 & 0 \\ 0 & r_2 \end{bmatrix}, \quad Q_T = \begin{bmatrix} q_{T1} & 0 \\ 0 & q_{T2} \end{bmatrix}$$

• make effect of $q_1 > q_2 > \frac{r_1}{r_2} r$

~~• ensure q_{T2} is large enough~~

(b) increasing the ~~r_1 and r_2~~ ^{r} to penalise control input more

Exercise 2 Consider a simplified model of a robotic arm in space, fixed to a spacecraft. The arm's dynamics can be approximated as a continuous-time, linear, time-invariant system:

$$\dot{x}(t) = Ax(t) + Bu(t),$$

where $x(t) \in \mathbb{R}^n$ is the state vector $x = [x_a, x_v]^\top$, with x_a a vector containing the joint angles and x_v a vector containing the angular velocities, and $u(t) \in \mathbb{R}^m$ is the control input, i.e. the torque applied to each joint.

The objective is to design a controller that moves the arm from an initial state x_0 to a desired final state $x_f = 0$ while avoiding a large obstacle. The obstacle's position imposes a state constraint the first element of x_a , which we can denote by $x_{a,1}$. More precisely, $x_{1,a}$ must not exceed a certain angular position, given by the constraint $x_{a,1}(t) <$

c , where c is a positive constant. Additionally, we want to minimise the control effort over an infinite horizon. This problem can be formulated as an infinite-horizon optimal control problem with a barrier function in the cost.

Consider the cost function:

$$J = \frac{1}{2} \int_0^\infty (x^\top Q x + u^\top R u + \phi(x_{a,1}(t))) dt,$$

where Q and R are positive definite matrices, and $\phi(x_1(t)) = -\mu \log(c - x_{a,1}(t))$ is a logarithmic barrier function with a small scalar constant $\mu > 0$.

- Explain the purpose of each term in the proposed cost function J . Make sure to address why the logarithmic barrier function is a suitable choice for this problem and what the role of the constant μ is.
- Discuss the trade-offs involved in selecting the values for the weight matrices Q and R and the constant μ . For example, what would happen if μ is chosen to be a very small or very large?

(a), (b) all the terms are running cost since it is an infinite-horizon optimal control problem.

$$Q = \begin{bmatrix} q_{aa} & q_{av} \\ q_{va} & q_{vv} \end{bmatrix} \text{ which can be simplified to } Q = \begin{bmatrix} q_a & 0 \\ 0 & q_v \end{bmatrix} \text{ to avoid}$$

cross-coupling terms. q_a and q_v are $N_a \times 1$ and $N_v \times 1$ vectors if x_a and x_v are $1 \times N_a$ and $1 \times N_v$ vectors, where q_a penalised joint angles error and q_v penalised angular velocities. Larger q , more precise control.

R penalised control input. Larger R , more energy efficient (less control).

Smaller $(c - x_{a,1}(t))$ is, i.e. robotic arm closer to obstacle, larger the term $-\ln(c - x_{a,1}(t))$ will be, therefore, μ is assigned to weightening the constraint, i.e. how ~~important the constraint is~~ far to obstacle.

(the constraint $x_1(t) < c$ is enforced)



Exercise 3 *The discrete-time dynamics of an inverted pendulum is given by*

$$x_{k+1} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ \frac{\Delta t}{M} \end{bmatrix} u_k$$

where $x = [x_p, x_v]^\top$, with x_p denoting the angular position of the arm and x_v it's velocity, M is the effective moment of inertia and Δt is the time step.

Suppose we want to design the control input u_k to keep the pendulum balanced in its upright position (i.e. $x_p = 0$) and at rest (i.e. $x_v = 0$) over a finite time horizon, while minimising energy consumption.

Consider the following finite-horizon optimal control problem with the cost function:

$$J = \frac{1}{2} x_N^\top Q_N x_N + \frac{1}{2} \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R u_k)$$

(a) *Discuss how the matrices Q , R , and Q_N should be assigned to achieve the following objectives:*

- *Strongly penalise the pendulum's angle (to keep it upright).*
- *Lightly penalise angular velocity (as long as the angle is near zero).*
- *Moderately penalise control effort (to prevent sudden, jerky movements).*

(b) *Discuss how the weighting matrices should be adjusted if the pendulum's battery is nearly depleted, making energy conservation the top priority. How would this affect the robot's balancing performance?*

(a) $Q = \begin{bmatrix} q_p & 0 \\ 0 & q_v \end{bmatrix}, R = r, Q_r = \begin{bmatrix} q_{rp} & 0 \\ 0 & q_{rv} \end{bmatrix}$

where

$$Q = \begin{bmatrix} \text{large} & 0 \\ 0 & \text{small} \end{bmatrix}, R = \text{moderate}, Q_r = \begin{bmatrix} \text{large} & 0 \\ 0 & \text{small} \end{bmatrix}$$

(b) Significantly increase the R , this will make the controller more conservative i.e. poor balancing performance.

Exercise 4 Consider a continuous-time, linear, time-invariant system,

$$\dot{x} = Ax,$$

with state $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Show that if all the eigenvalues of A are in the open left half-plane, the system is asymptotically stable.

The solution of $\dot{x} = Ax$ with initial condition $x(0) = x_0$ is given by

$$x(t) = e^{At} x_0$$

The system will be asymptotically stable if and only if $\lim_{t \rightarrow \infty} x(t) = 0$,

$$\text{i.e. } \lim_{t \rightarrow \infty} e^{At} x_0 = 0.$$

Consider the Jordan canonical form of A .

$$A = PJP^{-1} = P \begin{bmatrix} J_1 & & \\ & \ddots & \\ & & J_k \end{bmatrix} P^{-1} \quad \text{where } J_i = \begin{bmatrix} \lambda_i & 1 & & \\ & \lambda_i & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_i \end{bmatrix} = \lambda_i I + N_i$$

where N_i is nilpotent matrix.

Therefore,

$$e^{At} = Pe^{Jt}P^{-1} = P \begin{bmatrix} e^{J_1 t} & & \\ & \ddots & \\ & & e^{J_k t} \end{bmatrix} P^{-1}$$

$$\text{where } e^{J_i t} = e^{(\lambda_i I + N_i)t} = e^{\lambda_i t} e^{N_i t}$$

$$= e^{\lambda_i t} \left(I + N_i t + \frac{N_i^2 t^2}{2!} + \dots + \frac{N_i^{m_i-1} t^{m_i-1}}{(m_i-1)!} \right)$$

$$= e^{\lambda_i t} \begin{bmatrix} 1 & t & \frac{t^2}{2!} & \dots & \frac{t^{m_i-1}}{(m_i-1)!} \\ 0 & 1 & t & \dots & \frac{t^{m_i-2}}{(m_i-2)!} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & 1 & t \\ 0 & \dots & \dots & 0 & 1 \end{bmatrix}$$

So real part of λ_i must be negative.

Exercise 5 Consider a discrete-time, linear, time-invariant system,

$$x(k+1) = Ax,$$

with state $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Show that if all the eigenvalues of A are in the open unit disk, the system is asymptotically stable.

The solution of $x(k+1) = Ax$ with initial condition $x(0) = x_0$ is given by

$$x(k) = A^k x_0$$

The system will be asymptotically stable if and only if $\lim_{t \rightarrow \infty} x(t) = 0$,

i.e. $\lim_{t \rightarrow \infty} A^k x_0 = 0$.

Consider the Jordan canonical form of A ,

$$A = PJP^{-1} = P \begin{bmatrix} J_1 & & \\ & \ddots & \\ & & J_k \end{bmatrix} P^{-1} \quad \text{where } J_i = \begin{bmatrix} \lambda_i & 1 & & \\ & \lambda_i & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_i \end{bmatrix} = \lambda_i I + N_i$$

where N_i is nilpotent matrix.

Therefore,

$$A^k = PJ^kP^{-1} = P \begin{bmatrix} J_1^k & & \\ & \ddots & \\ & & J_k^k \end{bmatrix} P^{-1}$$

$$\text{where } J_i^k = \begin{bmatrix} \lambda_i^k & \binom{k}{1} \lambda_i^{k-1} & \dots & \binom{k}{m_i-1} \lambda_i^{k-(m_i-1)} \\ 0 & \lambda_i^k & \ddots & \vdots \\ \vdots & \ddots & \ddots & \binom{k}{1} \lambda_i^{k-1} \\ 0 & \dots & 0 & \lambda_i^k \end{bmatrix}$$

So λ_i must be in the open unit disk.